

Universidade Federal Rural de Pernambuco - UFRPE

Arquitetura e Organização de Computadores - 2026.2
Projeto 01 - Assembly MIPS e Simulador MARS
Prof. Vítor A Coutinho

INSTRUÇÕES

1. **Esta atividade compões nota de Projetos 1 (PE1) da 1a VA;**
2. A atividade deve ser feita em grupos de no máximo 4 pessoas;
3. Todas as questões devem ser implementadas utilizando Assembly MIPS e o simulador MARS;
4. Os arquivos entregues devem ser em formatos de código assembly *.asm, sendo 1 arquivo .asm ou .zip para cada questão de exercício e um arquivo .zip contendo um diretório para o projeto. Também é permitido os arquivos serem entregues através de um repositório do Github (OBS: este ŕepositório não pode ser atualizado após a data de entrega da atividade).
5. Os códigos devem estar **totalmente comentados**, explicando cada linha de código; cada arquivo deve apresentar um cabeçalho (utilizando comentários) contendo todas as informações (nome do grupo, atividade 1VA, disciplina, semestre letivo, breve descrição do conteúdo do arquivo, etc.);
6. O projeto deve ser desenvolvido em repositório do github, utilizando as boas práticas desse tipo de ferramenta.
7. A entrega será feita via Google classroom através de um link do repositório publico do github. O histórico de commits será observado e a organização do grupo, bem como a antecedência na execução do projeto, contará positivamente.
8. **O projeto deve ser desenvolvido sem o uso de compiladores e IA generativa,** uma vez que o propósito da atividade é desenvolver o pensamento computacional em linguagem assembly. O uso de linguagens de alto nível e linguagem natural fogem de tal objetivo.
9. **Não serão toleradas cópias/plágio da internet ou de outros colegas**, deste ou de outros semestres.
10. Qualquer irregularidade detectada implicará em penalização, podendo até gerar nota nula na 1VA. Evitem testar a capacidade de detecção do professor 😅

Não seguir as instruções pode incorrer em nota reduzida ou nula.

Projeto

O objetivo deste projeto é implementar um sistema de cadastro de pessoas e automóveis para um condomínio. Este sistema de cadastro deve ser operado através de um terminal (shell) que funciona como um interpretador de comandos de texto, ou seja, o sistema vai ficar constantemente checando por entradas de texto (string) e interpretando o que for recebido a partir de uma lista de comandos que o sistema deve ser capaz de executar. Os comandos que devem ser implementados serão descritos adiante na seção de requisitos de projeto.

ATENÇÃO: *é permitido utilizar o terminal padrão do MARS através das chamadas de syscall. Entretanto, o grupo que implementar o sistema utilizando o KEYBOARD MMIO e DISPLAY MMIO ganha (até)* ***1.0 ponto extra na nota do projeto!!***

Atividades práticas 1 a 4 valem (até) + 1.0 extra no projeto. ***Total de pontos extras no projeto: 1 (MMIO) + 1 (práticas) = 2***

Sugestão: *evite trabalhar com acentos e cedilhas por simplicidade.*

Os requisitos do projeto devem ser seguidos rigorosamente e são elucidados a seguir.

Requisitos de Projeto:

R1. O sistema deve ser capaz de registrar o nome completo dos moradores de cada apartamento do condomínio. O condomínio tem um total de 12 andares e 2 apartamentos por andar. Por regra do regimento do condomínio, não é permitido que cada apartamento tenha mais de 6 moradores por apartamento e o sistema não deve permitir que 7 ou mais moradores sejam cadastrados por apartamento.

R2. O sistema deve ser capaz de registrar o modelo e a placa do automóvel de cada apartamento. Por regra do condomínio, cada apartamento pode ter um carro na garagem ou até duas motos. Não é permitido ter um carro e uma ou mais motos para o mesmo AP. O sistema deve ser capaz de distinguir cadastro de carros e motos de modo a manter a contagem de acordo com a regra do condomínio (ex: se já houver um carro cadastrado, não pode ser cadastrado um novo carro ou uma nova moto).

R3. O sistema deve salvar todos os dados em um arquivo de modo que, caso o sistema seja reiniciado, os dados salvos anteriormente devem ser resgatados. Para tal, devem ser utilizados os serviços de arquivos do **syscall**. Sempre que o programa for inicializado, os dados salvos devem ser resgatados automaticamente.

R4. O sistema deve ser operado através através de um terminal que fica constantemente lendo entradas de texto (strings) e interpretando o que for recebido para executar comandos. O terminal deve aguardar uma quebra de linha “\n” para tentar interpretar o comando do usuário.

R5. A cada nova linha do terminal, uma string padrão deve ser impressa como “banner” na parte anterior ao campo que o usuário deve escrever um comando (similar a um terminal linux ou windows). O formato desta string deve ser “<iniciais_do_grupo>-shell>> “, em que <iniciais_do_grupo> deve ser substituído pelas iniciais dos nomes dos membros de cada grupo. Por exemplo, a string banner de um grupo genérico composto por Maria, João e José seria “MJJ-shell>> “. Segue um vídeo com este exemplo em anexo (**link**).

R6. O sistema deve ser capaz de distinguir um AP vazio (sem moradores) de um AP não vazio (que tem pelo menos 1 morador) para fornecer um panorama geral do condomínio de APs sendo usados e APs vazios.

R7. Todos os comandos devem ser interpretados através de uma string digitada pelo usuário e finalizados com uma quebra de linha (‘\n’). Alguns comandos podem ter opções e todas as opções devem ser iniciadas com a string “ --”, ou seja, espaço em branco (corresponde na Tabela ASCII ao byte 20, em hexadecimal e 32, em decimal) seguido de dois sinais de “menos” (byte 2d, em hexadecimal e 45 em decimal). Sempre que um comando não existente for executado, o terminal deve retornar uma mensagem “Comando invalido”. Todos os comandos da seguinte lista devem ser implementados:

cmd_1. `addMorador --<option1> --<option2>`

Este comando adiciona um morador a um apartamento especificado pela <option1>. O nome do morador é especificado pela <option2>. Se for adicionado um morador para um apartamento vazio, este deve mudar de estado para não vazio. Caso o número de moradores já esteja no limite, o terminal deve retornar uma mensagem “Falha: AP com numero max de moradores”. Caso seja digitado um AP inválido, deve retornar uma mensagem “Falha: AP invalido”.

Exemplo de uso: adicionar o morador Claude Shannon ao apartamento 1102:

```
addMorador --1102 --Claude Shannon
```

cmd_2. `rmvMorador --<option1> --<option2>`

Este comando remove um morador de um apartamento especificado pela <option1>. O nome do morador é especificado pela <option2>. Se for removido um morador e o apartamento não tiver mais moradores, este deve mudar de estado para vazio. Caso o apartamento fique vazio, os automóveis cadastrados devem ser automaticamente

excluídos. Caso o morador especificado não seja encontrado, o terminal deve retornar uma mensagem “Falha: morador nao encontrado”. Caso seja digitado um AP inválido, deve retornar uma mensagem “Falha: AP invalido”.

Exemplo de uso: remover o morador Joao Gomes ao apartamento 303:

```
rmvMorador --1102 --Claude Shannon
```

cmd_3. `AddAuto --<option1> --<option2> --<option3> --<option4>`

Este comando adiciona um automóvel a um apartamento especificado pela <option1>. O tipo de automóvel é especificado pela <option2>, podendo ser “m” para moto ou “c” para carro. O modelo do automóvel é especificado pela <option3> e a sua placa pela <option4>. Caso o número de automóveis já esteja no limite (1 carro ou 3 motos), o terminal deve retornar uma mensagem “Falha: AP com numero max de automóveis”. Caso seja digitado um AP inválido, deve retornar uma mensagem “Falha: AP invalido”. Caso seja digitado um tipo de automóvel inválido, deve retornar uma mensagem “Falha: tipo invalido”.

Exemplo de uso: adicionar um Fiat Uno de placa ABC1234 ao apartamento 303:

```
addAuto --303 --c --Fiat Uno --ABC1234
```

cmd_4. `rmvAuto --<option1> --<option2>`

Este comando remove um automóvel de um apartamento especificado pela <option1> . A placa do automóvel é especificada pela <option2>. Caso o automóvel especificado não seja encontrado, o terminal deve retornar uma mensagem “Falha: automóvel nao encontrado”. Caso seja digitado um AP inválido, deve retornar uma mensagem “Falha: AP invalido”.

Exemplo de uso: remover o Fiat Uno do apartamento 303:

```
rmvAuto --303 --ABC1234
```

cmd_5. `limparAp --<option1>`

Este comando exclui todos os moradores e automóveis cadastrados para o apartamento especificado pela <option1>, fazendo este apartamento ir para o estado vazio. Caso seja digitado um AP inválido, deve retornar uma mensagem “Falha: AP invalido”.

Exemplo de uso: limpar o apartamento 1001:

```
limparAp --1001
```

cmd_6. `infoAp --<option1>`

Este comando imprime na tela todas as informações cadastradas referente a um apartamento especificado pela <option1>. Caso seja digitado um AP inválido, deve retornar uma mensagem “Falha: AP invalido”. Caso o apartamento não tenha moradores, deve apresentar uma mensagem “Apartamento vazio”. Caso o apartamento tenha moradores, deve-se apresentar uma mensagem no padrão apresentado a seguir. Apenas as informações que forem cadastradas devem ser apresentadas. Se a <option1> for igual a “all”, deve exibir todos os apartamentos em sequência, em ordem crescente. Padrão:

(se vazio)
`AP: numero_do_ap`
Apartamento vazio

(se não vazio)
```
AP: numero_do_ap
Moradores:
Morador_1
(se tiver)        Morador_2
(se tiver)        Morador_3
(se tiver)        Morador_4
(se tiver)        Morador_5
(se tiver)        Morador_6
(se tiver)   Carro:
             Modelo_do_carro / placa_do_carro
(se tiver)   Moto:
             Modelo_da_moto_1 / placa_da_moto_1
             Modelo_da_moto_2 / placa_da_moto_2
             Modelo_da_moto_3 / placa_da_moto_3
```

cmd_7. `infoGeral`
Deve apresentar o panorama geral de apartamentos vazios e não vazios. Exemplo:

```
Não vazios:   300 (75%)
Vazios:       100 (25%)
```

cmd_8. `salvar`
Deve salvar todas as informações registradas em um arquivo externo. Cabe aos projetistas do grupo elaborar uma estrutura adequada para o formato do(s) arquivo(s).
cmd_9. `recarregar`

Recarrega as informações salvas no arquivo externo na execução atual do programa. Modificações não salvas serão perdidas e as informações salvas anteriormente recuperadas.

cmd_10. `formatar`

Apaga todas as informações da execução atual do programa, deixando todos os apartamentos vazios. Este comando não deve salvar automaticamente no arquivo externo, sendo necessário usar posteriormente o comando “salvar” para registrar a formação no arquivo externo.